PROJECT

Design.sv;

```
module traffic_signal_countdown #(
    parameter integer CLK_FREQ = 50_000_000,
    parameter integer GREEN_TIME = 10,
    parameter integer YELLOW_TIME = 4
)(
    input  logic clk,
    input  logic reset,

    output logic ns_red,
    output logic ns_yellow,
    output logic ns_green,

    output logic ew_red,
    output logic ew_yellow,
    output logic ew_green,

    output logic [6:0] seg
);

    // State Declaration

    typedef enum logic [1:0] {
        NS_GREEN,
        NS_YELLOW,
        EW_GREEN,
        EW_YELLOW
    } state_t;

    state_t current_state;
    state_t next_state;

 // Counters

    integer clock_count;
    integer countdown;

    integer next_countdown;

    // State Register and Timer

    always_ff @(posedge clk or posedge reset)
    begin
```

```
if (reset)
begin
   current_state <= NS_GREEN;

   clock_count <= 0;

   countdown <= GREEN_TIME - 1;
end

else
begin

   // Generate one second timing
   if (clock_count == CLK_FREQ - 1)
   begin
      clock_count <= 0;

      // If countdown has finished,
      // move to the next traffic state
      if (countdown == 0)
      begin
         current_state <= next_state;

         if (next_state == NS_GREEN)
            countdown <= GREEN_TIME - 1;

         else if (next_state == NS_YELLOW)
            countdown <= YELLOW_TIME - 1;

         else if (next_state == EW_GREEN)
            countdown <= GREEN_TIME - 1;

         else
            countdown <= YELLOW_TIME - 1;
      end

      else
      begin
         countdown <= countdown - 1;
      end
   end

   else
   begin
      clock_count <= clock_count + 1;
   end
```

```
    end
  end

// Next State Logic

  always_comb
  begin

    next_state = current_state;

    case (current_state)

      NS_GREEN:
      begin
        if (countdown == 0)
          next_state = NS_YELLOW;
      end

      NS_YELLOW:
      begin
        if (countdown == 0)
          next_state = EW_GREEN;
      end

      EW_GREEN:
      begin
        if (countdown == 0)
          next_state = EW_YELLOW;
      end

      EW_YELLOW:
      begin
        if (countdown == 0)
          next_state = NS_GREEN;
      end

      default:
      begin
        next_state = NS_GREEN;
      end

    endcase
  end

  // Traffic Light Output Logic
```

```
always_comb
begin

    // Default values
    ns_red = 0;
    ns_yellow = 0;
    ns_green = 0;

    ew_red = 0;
    ew_yellow = 0;
    ew_green = 0;


    case (current_state)

        // North-South Green

        NS_GREEN:
        begin
            ns_green = 1;
            ew_red = 1;
        end

// North-South Yellow

        NS_YELLOW:
        begin
            ns_yellow = 1;
            ew_red = 1;
        end

        // East-West Green

        EW_GREEN:
        begin
            ns_red = 1;
            ew_green = 1;
        end

  // East-West Yellow

        EW_YELLOW:
        begin
            ns_red = 1;
            ew_yellow = 1;
        end
```

```
        default:
        begin
            ns_red = 1;
            ew_red = 1;
        end

    endcase
end


// 7 Segment Display
//
// Display is active LOW.
// 0 = segment ON
// 1 = segment OFF


always_comb
begin

    case (countdown)

        0:
            seg = 7'b1000000;

        1:
            seg = 7'b1111001;

        2:
            seg = 7'b0100100;

        3:
            seg = 7'b0110000;

        4:
            seg = 7'b0011001;

        5:
            seg = 7'b0010010;

        6:
            seg = 7'b0000010;

        7:
            seg = 7'b1111000;
```

```
            8:
                seg = 7'b0000000;

            9:
                seg = 7'b0010000;

            default:
                seg = 7'b1111111;

        endcase

    end

endmodule
```

Testbench.sv

```
`timescale 1ns/1ps

module tb_traffic_signal;

    // Testbench Signals
    logic clk;
    logic reset;

    logic ns_red;
    logic ns_yellow;
    logic ns_green;

    logic ew_red;
    logic ew_yellow;
    logic ew_green;

    logic [6:0] seg;

    // Clock Generation
    initial
    begin
        clk = 0;

        forever
            #5 clk = ~clk;
    end


    // Device Under Test
    //
    // CLK_FREQ = 10 means:
```

```
// 10 clock cycles = 1 simulated second
//
// Green = 10 seconds
// Yellow = 4 seconds

traffic_signal_countdown #(
    .CLK_FREQ(10),
    .GREEN_TIME(10),
    .YELLOW_TIME(4)
)
dut
(
    .clk(clk),
    .reset(reset),

    .ns_red(ns_red),
    .ns_yellow(ns_yellow),
    .ns_green(ns_green),

    .ew_red(ew_red),
    .ew_yellow(ew_yellow),
    .ew_green(ew_green),

    .seg(seg)
);


// Test Sequence

initial
begin
$dumpfile("traffic_signal.vcd");
$dumpvars(0, tb_traffic_signal);

    // Start with reset
    reset = 1;

    #20;

    reset = 0;


    // Run the complete traffic sequence
    // 10 sec NS Green
    // 4 sec NS Yellow
    // 10 sec EW Green
    // 4 sec EW Yellow
```

```
        //
        // Total = 28 seconds
        //
        // Simulation clock:
        // 10 clock cycles = 1 second
        //
        // Clock period = 10 ns
        //
        // Therefore approximately:
        // 28 x 10 x 10 ns = 2800 ns

        #3000;

        // Test reset again
        reset = 1;

        #20;

        reset = 0;

        #500;


        $finish;
    end

    // Display Values in Simulation Console

    always @(posedge clk)
    begin

        $display(
            "Time = %0t | State = %0d | Countdown = %0d | NS: R=%b Y=%b G=%b
| EW: R=%b Y=%b G=%b",
            $time,
            dut.current_state,
            dut.countdown,
            ns_red,
            ns_yellow,
            ns_green,
            ew_red,
            ew_yellow,
            ew_green
        );
    end
endmodule
```